

Documentación de Trabajo Práctico Integrador - UNQ Shop

Integrantes:

- Mamani Yampa, Rocio Belen (rocidxs@gmail.com)
- Cardozo, Marco Antonio (marcocardozo325@gmail.com)
- Zevallos Victorio, Luis Enrique (luis.e.zevallos@hotmail.com)

Repositorio de GitHub: <https://github.com/cardozo-marco/POO2UNQShop>

Decisiones de Diseño

2.1 Catálogo de productos

Para modelar los ítems comercializables por la tienda se implementó el **Patrón Composite**. Se definió una interfaz común ItemCatalogo que es implementada tanto por los productos individuales (Producto) como por las agrupaciones de estos (Paquete). Esta decisión de diseño permite que el Pedido y otros subsistemas traten a cualquier ítem de forma polimórfica y uniforme, sin importar si es un ítem simple o un conjunto complejo con descuentos aplicados. De esta forma, el sistema puede calcular precios base y finales delegando la responsabilidad de forma recursiva en cada componente, sin necesidad de consultar el tipo concreto del ítem.

2.2 Ciclo de vida del pedido

En este punto se debía modelar el ciclo de vida de un pedido, teniendo en cuenta que las operaciones permitidas cambian según el estado en el que se encuentre. Por ejemplo, un pedido en estado **Borrador** permite agregar o quitar ítems, mientras que un pedido **Enviado** ya no admite modificaciones. Además, las operaciones inválidas debían lanzar una excepción de dominio específica en lugar de una excepción genérica.

Haber concentrar toda la lógica del ciclo de vida en la clase Pedido mediante múltiples estructuras condicionales (if o switch) habría dificultado el mantenimiento del sistema y obligado a modificar la clase cada vez que se agregara un nuevo estado o cambiarán las reglas de negocio. En su lugar, cada estado fue modelado como una clase independiente que implementa el comportamiento correspondiente a ese momento del ciclo de vida utilizando el **Patrón State**. De esta manera, Pedido únicamente mantiene una referencia a su estado actual y delega en él la ejecución de cada operación. Cuando el pedido cambia de estado, simplemente reemplaza esa referencia por una nueva instancia del estado correspondiente, logrando que su comportamiento varíe dinámicamente sin modificar la implementación de la clase principal.

Generación de nota de crédito

Se decidió que la responsabilidad permanezca dentro del comportamiento del estado que realiza la cancelación, ya que es cada estado quien conoce las reglas de negocio que deben aplicarse. Por ejemplo, cuando un pedido se cancela desde **Confirmado** o **En Preparación** se repone el stock y se reembolsa tanto el costo de los productos como el del envío, mientras que si la cancelación ocurre desde **Enviado** únicamente se reintegra el valor de los productos. Los estados que no deben generar una nota de crédito simplemente realizan la cancelación o impiden la operación.

Patron State : Permite que un objeto modifique su comportamiento cuando cambia su estado interno, delegando la lógica específica de cada estado en clases independientes y evitando el uso de grandes estructuras condicionales.

Roles:

- **Context:** Pedido. Mantiene una referencia al estado actual (EstadoPedido) y delega en él todas las operaciones relacionadas con el ciclo de vida del pedido.
- **State:** EstadoPedido. Define la interfaz común con las operaciones que pueden realizarse sobre un pedido (confirmar, preparar, enviar, entregar, cancelar, agregarItem y quitarItem).
- **ConcreteState:** EstadoBorrador, EstadoConfirmado, EstadoEnPreparacion, EstadoEnviado, EstadoEntregado y EstadoCancelado. Cada una implementa el comportamiento correspondiente al estado que representa, permitiendo o rechazando operaciones, realizando las transiciones válidas y aplicando las reglas de negocio asociadas, como la actualización del stock o la emisión de la nota de crédito.

2.3 Métodos de envío:

En esta parte se tenían que manejar distintos tipos de envío (Estándar, Express y Retiro en Sucursal), donde cada uno cuenta con su propia forma de calcular los costos y estimar los días de demora. En lugar de usar muchos condicionales dentro de la clase Pedido (lo cual haría el código muy difícil de mantener y violaría el principio Open/Closed), decidimos encapsular cada cálculo en su propia clase separada.

Un detalle de la implementación es que usamos el tipo de dato primitivo float en vez de double. Hicimos esto a propósito para respetar la firma que nos dio la cátedra para la API del correo (float CorreoArgentina.estimarEnvio(float peso, Direccion direccionEnvio;)) y así no tener problemas de conversión de tipos. Además, decidimos pasarle la instancia entera del Pedido (el contexto) por parámetro a la estrategia para que cada método de envío le pida solo los datos que realmente necesita (como el peso, el precio total o la dirección), logrando así que las clases queden desacopladas y no reciban datos de más.

Patrón de Diseño: Strategy. Este patrón permite definir una familia de algoritmos, encapsular cada uno de ellos y hacerlos intercambiables, permitiendo que el algoritmo varíe independientemente del cliente que lo utiliza.

Roles:

- **Strategy:** MetodoEnvio. Es la interfaz común que declara las operaciones calcularCosto(Pedido) y estimarDiasEntrega(Pedido).
- **ConcreteStrategy:** EnvioEstandar, EnvioExpress y RetiroEnSucursal. Implementan los algoritmos específicos utilizando la interfaz de la estrategia.
- **Context:** Pedido. Mantiene una referencia a un objeto MetodoEnvio y delega en él la responsabilidad de calcular el costo y los días de demora.

2.4 Procesamiento de pagos

Para la validación y el cobro de los pedidos se optó por utilizar el Patrón Template Method. Se creó una clase abstracta MetodoPago que define el esqueleto del algoritmo de pago (verificar fondos, aplicar recargos o descuentos, y registrar la transacción). Las variantes concretas, como TarjetaCredito o TransferenciaBancaria, implementan únicamente los pasos específicos que difieren entre sí. Esto promueve la reutilización de código en la lógica en común y facilita enormemente la incorporación de nuevos medios de pago en el futuro sin alterar el flujo principal de cobro.

2.5 Notificaciones del pedido:

El TP nos pedía que, cuando un pedido cambie de estado, varios sistemas externos (como el generador de facturas, el de fidelización o el notificador de emails) hagan diferentes tareas de forma automática. Pero la clase Pedido no tenía que enterarse de la existencia de ellos para mantener el sistema lo más desacoplado posible.

Para lograr esto, armamos un sistema de suscripción y notificación. Básicamente, el Pedido mantiene una lista de observadores (las interfaces abstractas) y, cuando nota un cambio en su estado interno, recorre esa lista avisándoles a todos lo que pasó. De esta manera, el día de mañana podemos agregar nuevos subsistemas de notificación o sacar los actuales sin necesidad de modificar el código de la clase Pedido.

Patrón de Diseño: Observer. Lo elegimos porque permite definir una dependencia de "uno a muchos" entre objetos, donde si el estado de un objeto cambia, todos los objetos que dependen de él reciben una notificación y se actualizan solos.

Roles:

- Subject: Pedido (o una clase abstracta de la que herede). Es el que conoce a su lista de observadores y proporciona la interfaz para que éstos se suscriban o desuscriban.
- Observer: La interfaz común (PedidoObserver), que define el método para que los objetos se actualicen al recibir el aviso de cambio.
- ConcreteObserver: Los distintos subsistemas reales (el notificador de Mail, el Facturador o el sistema de Fidelización). Éstos guardan su propio estado y ejecutan su lógica particular cuando la interfaz les avisa del cambio.

2.6 Búsqueda en el catálogo

Se debía implementar un sistema de búsqueda que permitiera encontrar productos y paquetes aplicando distintos criterios. La búsqueda debía soportar tanto criterios simples (como nombre, categoría, precio máximo o disponibilidad) como consultas mucho más complejas, combinando esos criterios mediante operadores lógicos AND, OR y NOT, con la posibilidad de anidarlos indefinidamente.

Para implementar la solución modelamos cada criterio como un objeto independiente que conoce la forma de evaluar un ítem del catálogo. De esta manera, el catálogo únicamente recibe un criterio y delega completamente en él la responsabilidad de decidir si un elemento satisface o no la búsqueda.

Se logró que los operadores lógicos puedan combinar criterio simples y otros operadores de forma uniforme utilizando el Patrón Composite. Así, tanto un criterio simple como uno compuesto implementan la misma interfaz (CriterioBusqueda), permitiendo construir

consultas de cualquier profundidad sin modificar el módulo de búsqueda. De esta manera la búsqueda queda distribuida entre los distintos criterios y el catálogo permanece desacoplado de la lógica de evaluación.

Patrón Composite: Permite representar estructuras jerárquicas donde los objetos simples y los compuestos pueden tratarse de manera uniforme. En nuestro caso, hace posible construir expresiones de búsqueda complejas combinando criterios simples y operadores lógicos sin distinguir entre ellos.

Roles:

- **Component:** CriterioBusqueda. Es la interfaz común que declara la operación `satisface(ItemCatalogo)` que todos los criterios deben implementar.
- **Leaf:** CriterioPorNombre, CriterioPorCategoria, CriterioPorPrecioMaximo y CriterioPorDisponibilidad. Representan los criterios simples, encargados de evaluar una única condición sobre un ítem del catálogo.
- **Composite:** CriterioAND, CriterioOR y CriterioNOT. Representan los criterios compuestos que contienen uno o varios objetos CriterioBusqueda y combinan sus resultados mediante operadores lógicos.
- **Client:** Catalogo (o cualquier cliente que realice una búsqueda). Recibe un único objeto CriterioBusqueda sin importar si representa un criterio simple o una composición arbitrariamente compleja.

2.8 Reportes:

Para esta parte, teníamos que poder exportar los reportes del negocio (por ejemplo, los productos más vendidos) en diferentes formatos visuales, como texto plano, CSV y HTML. Si metíamos el código de exportación directamente dentro de los ítems del catálogo (como Producto o Paquete), íbamos a contaminar nuestro modelo de dominio puro con lógica de presentación visual. Y también, nos iba a obligar a modificar todas esas clases cada vez que nos pidieran un formato de reporte nuevo.

La solución fue sacar esa lógica de formato hacia objetos "visitantes" completamente externos. El sistema va iterando por los elementos y recolectando los datos usando *double-dispatch* (mediante el método `aceptar()`). Así, la operación final que se ejecuta depende de dos cosas al mismo tiempo: de qué formato queremos exportar (el visitante) y de qué ítem particular del catálogo estamos leyendo en ese momento.

Patrón de Diseño: Visitor. Representa una operación sobre los elementos de una estructura, permitiéndonos inventar nuevas operaciones (como nuevos reportes o formatos) sin tener que cambiar las clases de los elementos que estamos visitando.

Roles:

- **Visitor:** Nuestra interfaz FormateadorReporte, que declara una operación particular (`visitar...()`) para cada clase concreta que existe en nuestro catálogo.
- **ConcreteVisitor:** Las clases que arman el documento final, como ReporteTexto, ReporteCSV y ReporteHTML. Éstas van acumulando el texto o el estado a medida que recorren la estructura.
- **Element:** La interfaz base de nuestro catálogo (ItemCatalogo), que define la operación principal `aceptar(Visitor)`.

- **ConcreteElement:** Nuestras clases de dominio Producto y Paquete, que implementan el aceptar() invocando la operación de visita que les corresponde dentro del visitante.
- **ObjectStructure:** El registro de ventas o el catálogo que nos sirve para enumerar los ítems y permitirle al visitante acceder a ellos uno por uno.

2.X Gestión principal de la tienda (Shop)

Luego de haber diseñado los distintos subsistemas de manera independiente y desacoplada (catálogo, pedidos, envíos y pagos), se implementó el **Patrón Facade** a través de la clase ShopGestionada para unificar la interacción con el sistema. Esta clase actúa como un punto de entrada único que orquesta todas las operaciones de alto nivel (como registrar un cliente, crear un pedido completo o solicitar la generación de reportes). Se decidió centralizar esta responsabilidad para ocultar la complejidad estructural del dominio subyacente, proveyendo a cualquier cliente externo una interfaz de uso sencilla, cohesiva y libre de acoplamiento.